STUDENT COURSE NOTES

Python 3.12 Asynchronous Concurrency

Professor: Dr. Samriddhi Khade • College: DPU • Stack: Python 3.12

Slide 2: Structured Concurrency with TaskGroup

Concept Explanation: Core conceptual mechanism.

Why It Matters in Production: Critical for high performance production systems.

Common Pitfall: Unchecked thread contention or resource leakage.

```python
import asyncio

async def fetch_service(svc_id: int) -> str:
    await asyncio.sleep(0.05)
    return f"Service {svc_id} ready"

async def main():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_service(1))
        t2 = tg.create_task(fetch_service(2))
    return t1.result(), t2.result()
```

Captured Output:
```
Process executed cleanly with 0 exit code.
```

Slide 3: AsyncIO Event Loop Workflow

Concept Explanation: Core conceptual mechanism.

Why It Matters in Production: Critical for high performance production systems.

Common Pitfall: Unchecked thread contention or resource leakage.

Slide 4: Best Practices in AsyncIO Systems

Concept Explanation: Core conceptual mechanism.

Why It Matters in Production: Critical for high performance production systems.

Common Pitfall: Unchecked thread contention or resource leakage.